Probeklausur: Mikrocomputertechnik 1

Bearbeitungszeit: 90 Minuten · Gesamtpunktzahl: 90 Punkte

Prof. Dr.-Ing. Daniel Klünder

Beantworten Sie die nachfolgenden Fragen im Multiple-Choice-Format. Bei jeder Frage können eine, mehrere oder alle Antwortmöglichkeiten richtig sein. Jede Teilaufgabe wird mit einem Punkt bewertet – dieser wird nur bei vollständig korrekter Antwort vergeben.

Musterlösung mit Bewertungshinweisen

Name:

_____

Matrikelnummer:

_____

Kurs / Studiengruppe:

_____

Hilfsmittel: Das offizielle RISC-V Green Card / Reference Sheet wird ausgeteilt. Alle Aufgaben sind auf Papier ohne Simulator oder Compiler lösbar.

Stoffumfang: Vorlesungen 1–7 und deren Wiederholung in Einheit 9.

Markieren Sie bei den Fragen 1–20 jeweils alle richtigen Optionen. Eine vollständig richtige Auswahl erhält einen Punkt, jede andere Auswahl null Punkte. Es gibt keine Teilpunkte und keinen Punktabzug. Ab Aufgabe 2 sind Rechenwege beziehungsweise Begründungen anzugeben; dort sind Teilpunkte möglich.

Aufgabe	Inhalt	Punkte	Richtzeit	erreicht
1	20 Multiple-Choice-Fragen, Vorlesungen 1–7	20	20 min	
2	Zahlendarstellung, Logik und Register	10	10 min	
3	Array und Schleife in RV32I	16	16 min	
4	Funktionsaufruf und Stacktrace	10	10 min	
5	Datenpfad und Steuerwerk	12	12 min	
6	Pipeline und Laufzeit	12	12 min	
7	Maschinencode und Bytefolge	10	10 min	
	Summe	**90**	**90 min**	

Die Richtzeiten umfassen Lesen und Bearbeiten und dienen der Zeiteinteilung.

Gemeinsame Annahmen: RV32I ohne komprimierte Instruktionen, 32-Bit-Register, Little Endian und Byteadressierung. Echte Instruktionen sind vier Byte lang. Registerarithmetik behält die unteren 32 Bit. Normale Funktionsaufrufe verwenden die ILP32-Konvention: skalare Ergebnisse in `a0`, Stackzeiger während standardkonformer Funktionen auf 16 Byte ausgerichtet. Die Arrayelemente sind 32-Bit-Wörter. Datenadressen und Speicherbereiche sind gültig, soweit nicht ausdrücklich nach einer Bereichsgrenze gefragt wird.

Datenpfad und Pipeline sind die jeweils ausdrücklich angegebenen Lehrmodelle. Zeit- und Adresswerte sind Aufgabenannahmen, keine gemessenen Eigenschaften eines Prozessors oder Simulators.

Aufgabe 1: Multiple Choice (20 Punkte)

Fragen 1–4

Frage 1: ISA und Umsetzung (1 Punkt)

Zwei Prozessoren führen denselben RV32I-Code korrekt aus, benötigen dafür aber verschieden viele Takte. Welche Aussagen treffen zu?

- ☐ A: Beide können dieselbe ISA mit unterschiedlichen Mikroarchitekturen implementieren.
- ☐ B: Die unterschiedliche Taktzahl verlangt unterschiedliche RV32I-Instruktionscodierungen.
- ☐ C: Die Laufzeit lässt sich allein aus den Assemblerzeilen bestimmen.
- ☐ D: Die ISA beschreibt das sichtbare Verhalten, nicht jede interne Taktfolge.

Frage 2: Bitmuster und Zahlenwert (1 Punkt)

Das achtbittige Muster `1111 1011` wird ohne Änderung der Bits betrachtet. Welche Aussagen treffen zu?

- ☐ A: Als vorzeichenloser Wert stellt es die Zahl 251 dar.
- ☐ B: Als Zweierkomplementwert stellt es die Zahl −5 dar.
- ☐ C: Mit Vorzeichenerweiterung wird daraus `0xFFFFFFFB`.
- ☐ D: Mit Nullerweiterung wird daraus `0x000000FB`.

Frage 3: Logik und gespeicherter Zustand (1 Punkt)

Ein Multiplexer und ein positiv flankengetriggertes Register werden ohne Reset betrachtet. Welche Aussagen treffen zu?

- ☐ A: Ein Multiplexer übernimmt seine neue Auswahl erst an der nächsten Taktflanke.
- ☐ B: Ein Multiplexer kann seinen Ausgang nach einer Auswahländerung zwischen Flanken ändern.
- ☐ C: Das Register folgt bei dauerhaft hohem Takt jeder Änderung seines Dateneingangs.
- ☐ D: Ohne neue aktive Flanke bleibt der gespeicherte Registerwert erhalten.

Frage 4: Register und Immediates (1 Punkt)

Für RV32I ohne Erweiterungen: Welche Aussagen treffen zu?

- ☐ A: Nach `addi x0, x0, 7` enthält das Register `x0` den Wert sieben.
- ☐ B: Das Immediate −2048 ist bei `addi` direkt codierbar.
- ☐ C: Die Anweisung `addi t0, t1, 2048` verwendet ein zulässiges Immediate.
- ☐ D: Eine Addition über die 32-Bit-Grenze hinaus löst einen Überlauf-Trap aus.

Fragen 5–8

Frage 5: Instruktionsformate (1 Punkt)

Vergleichen Sie `sub t0, t1, t2` und `sw t0, -4(t1)`. Welche Aussagen über die Maschinenwörter treffen zu?

- ☐ A: Das R-Format von `sub` enthält die Felder `rd`, `rs1` und `rs2`.
- ☐ B: Das S-Format von `sw` benennt im Feld `rd` das gespeicherte Register.
- ☐ C: Bei `sw` ist das Immediate auf zwei Teile des Befehlsworts verteilt.
- ☐ D: Bei `sw` bezeichnet `rs2` die Quelle der zu speichernden Daten.

Frage 6: Speicher und Erweiterung (1 Punkt)

Das Wort `0x00000080` liegt ab Adresse `0x2000` in Little Endian. Die Zugriffe sind gültig. Welche Aussagen treffen zu?

- ☐ A: An Adresse `0x2000` liegt das Byte `0x00`.
- ☐ B: `lb t0, 0(a0)` mit `a0=0x2000` liefert `0xFFFFFF80`.
- ☐ C: `lbu t0, 0(a0)` mit `a0=0x2000` liefert `0x00000080`.
- ☐ D: Das Basisregister `a0` wird durch den Load um ein Byte erhöht.

Frage 7: Bedingte Sprünge (1 Punkt)

Vor dem Branch gilt `t0=0xFFFFFFFF` und `t1=1`. Welche Aussagen treffen zu?

- ☐ A: `blt t0, t1, ziel` nimmt den Branch.
- ☐ B: `bltu t0, t1, ziel` nimmt den Branch.
- ☐ C: `beq t0, t1, ziel` nimmt den Branch.
- ☐ D: `bgeu t0, t1, ziel` nimmt den Branch.

Frage 8: Indirekter Aufruf (1 Punkt)

Die echte Vier-Byte-Instruktion `jalr ra, 5(ra)` steht bei `PC=0x1000`; vorher enthält `ra` den Wert `0x2000`. Welche Aussagen treffen zu?

- ☐ A: Die Berechnung des Ziels verwendet bereits den neuen Linkwert `0x1004`.
- ☐ B: Nach der Instruktion enthält `ra` den Linkwert `0x1004`.
- ☐ C: Die nächste Instruktion liegt an Adresse `0x2004`.
- ☐ D: Vor dem Sprung wird der alte Inhalt von `ra` automatisch gesichert.

Fragen 9–12

Frage 9: Aufrufkonvention (1 Punkt)

Eine normal aufgerufene ILP32-Funktion verändert `s0` und ruft mit `jal ra, helfer` eine weitere Funktion auf. Welche Aussagen treffen zu?

- ☐ A: Sie muss ihren eingehenden `s0`-Wert vor der eigenen Rückkehr wiederherstellen.
- ☐ B: Der Helfer muss alle vom Aufrufer noch benötigten `t`-Register erhalten.
- ☐ C: Das Pseudokommando `ret` stellt den alten Stackpointer automatisch wieder her.
- ☐ D: Sie muss ihre äußere Rückkehradresse über den Helferaufruf hinweg erhalten.

Frage 10: Load im Datenpfad (1 Punkt)

Der Single-Cycle-Datenpfad führt `lw t0, 12(t1)` aus. Welche Aussagen treffen zu?

- ☐ A: Die ALU addiert zum Basiswert den vorzeichenerweiterten Byteoffset zwölf.
- ☐ B: Der zweite ALU-Operand ist der aus `t0` gelesene alte Registerwert.
- ☐ C: Die gelesenen Speicherdaten bilden den Rückschreibwert für `t0`.
- ☐ D: Der Wert aus `t0` wird bei diesem Befehl in den Datenspeicher geschrieben.

Frage 11: Branch-Entscheidung im Datenpfad (1 Punkt)

Ein `beq` steht bei `PC=0x8000`; ImmGen liefert den fertigen Byteoffset 20 und die Operanden sind gleich. Welche Aussagen treffen zu?

- ☐ A: Der nächste PC ist wegen `PC + 4 + Offset` gleich `0x8018`.
- ☐ B: Die ALU-Subtraktion liefert für den Vergleich `Zero=1`.
- ☐ C: Die Auswahl `PCSrc` entsteht aus `Branch` und `Zero`.
- ☐ D: Der nächste PC ist wegen `PC + Offset` gleich `0x8014`.

Frage 12: Hauptsteuerung und ALU-Steuerung (1 Punkt)

Im Kursmodell werden `add` und `sub` mit gleichem Opcode, aber unterschiedlichen gültigen Funktionsfeldern ausgeführt. Welche Aussagen treffen zu?

- ☐ A: Die beiden Befehle benötigen unterschiedliche Werte des Signals `RegWrite`.
- ☐ B: Die ALU-Detailsteuerung unterscheidet sie anhand der Funktionsfelder.
- ☐ C: Die vier Bits von `ALUControl` sind ein vorgeschriebenes RISC-V-Instruktionsfeld.
- ☐ D: Der siebenbittige Opcode unterscheidet bereits beide Rechenoperationen.

Fragen 13–16

Frage 13: Auswahl und Schreibfreigabe (1 Punkt)

Ein korrekt gesteuerter `sw t0, 4(t1)` hat `RegWrite=0`. Welche Aussagen über `MemtoReg` treffen zu?

- ☐ A: Der Steuerungswert X kennzeichnet einen zwingend hochohmigen elektrischen Pegel.
- ☐ B: Die Werte null und eins lassen den sichtbaren Zustand dieses Stores unverändert.
- ☐ C: Die Auswahl des Rückschreibwerts ist wegen der gesperrten Registerübernahme unerheblich.
- ☐ D: Auch bei `RegWrite=1` kann keine Rückschreibquelle ein Register verändern.

Frage 14: Laufzeitvergleich (1 Punkt)

Zwei Ausführungen haben jeweils `N=100` Maschineninstruktionen. A hat `CPI=1` und eine Taktperiode von `4 ns`, B hat `CPI=2` und `1 ns`. Welche Aussagen treffen zu?

- ☐ A: Die Ausführung A benötigt insgesamt eine Zeit von 400 ns.
- ☐ B: Die Ausführung A ist wegen ihres kleineren CPI schneller.
- ☐ C: Die Ausführung B benötigt insgesamt eine Zeit von 200 ns.
- ☐ D: Der Quotient Zeit A durch Zeit B beträgt zwei.

Frage 15: Sprungvorhersage (1 Punkt)

Ein isolierter Branch hat die tatsächliche Folge T, T, N (genommen, genommen, nicht genommen). Ein Einbitprädiktor startet mit N und speichert nach jedem Branch dessen tatsächliche Richtung als nächste Vorhersage. Welche Aussagen treffen zu?

- ☐ A: Die Richtungen werden in der Reihenfolge N, T, T vorhergesagt.
- ☐ B: Die Richtungen werden in der Reihenfolge N, N, T vorhergesagt.
- ☐ C: Die Vorhersage ist bei allen drei Ausführungen unzutreffend.
- ☐ D: Die nächste Vorhersage nach dem letzten Branch ist genommen.

Frage 16: Logisch und arithmetisch schieben (1 Punkt)

Die folgenden RV32I-Befehle werden unabhängig voneinander mit `t0=0x80000008` und `t2=35` betrachtet. Welche Aussagen treffen zu?

- ☐ A: `srai t1, t0, 3` ergibt das Muster `0xF0000001`.
- ☐ B: `srli t1, t0, 3` ergibt das Muster `0xF0000001`.
- ☐ C: `srl t1, t0, t2` verschiebt mit dem gesamten Betrag 35.
- ☐ D: `srl t1, t0, t2` verwendet einen Schiebebetrag von drei.

Fragen 17–20

Frage 17: Load-Use mit Forwarding (1 Punkt)

Eine anfangs leere Fünfstufen-Kurspipeline mit EX-Forwarding führt `lw t0, 0(s0)` und direkt danach `sub a0, t0, t1` aus. `s0` zeigt auf ein gültiges Wort. Welche Aussagen treffen zu?

- ☐ A: Der geladene Wert steht bereits nach EX zur Weiterleitung bereit.
- ☐ B: Ein Datenwartezyklus genügt für die korrekte Ausführung.
- ☐ C: Vom ersten IF bis zum letzten WB werden sieben Takte benötigt.
- ☐ D: Die Bubble erhöht die Zahl abgeschlossener Programmbefehle auf drei.

Frage 18: Der jüngste Produzent gewinnt (1 Punkt)

Die Fünfstufen-Kurspipeline führt mit Forwarding `add t0, t1, t2`, `addi t0, t0, 4` und `sub a0, t0, t4` aus. Anfangs gelten `t1=6`, `t2=5` und `t4=3`. Welche Aussagen treffen zu?

- ☐ A: Der Verbraucher `sub` muss das ältere Ergebnis elf verwenden.
- ☐ B: Für `sub` hat der Treffer aus EX/MEM Vorrang vor MEM/WB.
- ☐ C: Der letzte Befehl schreibt den Wert zwölf nach `a0`.
- ☐ D: `add` und `addi` erzeugen verschiedene Versionen von `t0`.

Frage 19: Benutzte Quellregister (1 Punkt)

Auf `lw t0, 0(s0)` folgt `addi a0, x0, 5`; die Adresse ist gültig. `t0` heißt auch `x5`. Beim `addi` sind die Instruktionsbits 24–20 gleich `00101`. Welche Aussagen treffen zu?

- ☐ A: Zwischen diesen Befehlen besteht keine Registerdatenabhängigkeit.
- ☐ B: Die Bits `00101` bezeichnen beim `addi` das Quellregister `x5`.
- ☐ C: Ein Wartezyklus ist allein wegen dieser Bitübereinstimmung nötig.
- ☐ D: `addi` liest `x0` und den Inhalt von `t0` als seine beiden Quellen.

Frage 20: Flush bei genommenem Branch (1 Punkt)

Ein `beq` wird in EX als genommen erkannt. Bis dahin wurde sequenziell geholt: Ein jüngerer `sw` steht in ID, ein jüngerer `addi` in IF. Welche Aussagen treffen zu?

- ☐ A: Beide jüngeren Einträge müssen ihre Gültigkeit verlieren.
- ☐ B: Der verworfene Store darf den Datenspeicher nicht verändern.
- ☐ C: Die zwei verworfenen Einträge erhöhen die abgeschlossene Befehlszahl N.
- ☐ D: Ein Flush hält die jüngeren Befehle bis zur nächsten Taktflanke nur an.

Musterlösung zu Aufgabe 1

Bewertung: Je Frage ein Punkt ausschließlich für die exakte Auswahl aller richtigen und keiner falschen Optionen. Keine Teilpunkte, kein Punktabzug. Die Zahl richtiger Antworten ist nicht vorgegeben.

1. A, D — A und D: Die Instruction Set Architecture (ISA) legt die architektonische Befehlswirkung fest. B ist falsch: Andere Taktkosten erfordern keine andere Codierung. C ist falsch: Befehlszahl, Taktkosten und Taktperiode hängen auch von Expansion und Umsetzung ab.

2. A, B, C, D — Alle vier Optionen: Das Bitmuster hat unsigned den Wert 251, signed den Wert $251 - 256 = -5$. Vorzeichenerweiterung übernimmt das gesetzte Vorzeichenbit; Nullerweiterung füllt die oberen Stellen mit null. Die Interpretation und die Erweiterungsart sind unterschiedliche Entscheidungen.

3. B, D — B und D: Der Multiplexer ist kombinatorisch und reagiert nach seiner Laufzeit. Das Register übernimmt bei der aktiven Flanke, sofern die Freigabe gesetzt ist. A verwechselt Auswahl und Speicherung; C beschreibt ein pegelgesteuertes statt eines flankengetriggerten Speicherelements. Setup und Hold seien eingehalten.

4. B — Nur B: Das signed 12-Bit-Immediate reicht von -2048 bis 2047. Schreibzugriffe auf x0 werden verworfen; 2048 liegt außerhalb des Bereichs. RV32I-Ganzzahladdition behält die unteren 32 Bit und erzeugt keinen Überlauf-Trap.

5. A, C, D — A, C und D: sub liest zwei Register und schreibt rd. sw hat zwei Registerquellen für Basis und Daten und kein rd-Feld. Die Positionen 11:7 enthalten beim Store den unteren Immediate-Teil; sie bezeichnen dort kein Zielregister.

6. B, C — B und C: Das niedrigstwertige Byte 0x80 liegt an der kleinsten Adresse. lb erweitert sein Bit 7 als Vorzeichen, lbu erweitert mit Nullen. A vertauscht die Byteordnung; D unterstellt eine automatische Adressänderung, die diese Loads nicht ausführen.

7. A, D — A und D: Signed ist t0 gleich -1 und damit kleiner als 1. Unsigned ist dasselbe Muster 4294967295 und damit größer oder gleich 1. B verwendet die falsche unsigned Relation; C ist falsch, weil die Registermuster verschieden sind.

8. B, C — B und C: jalr liest die alte Quelle, berechnet 0x2000 + 5 = 0x2005 und löscht Bit 0, also Ziel 0x2004. Der neue Linkwert ist der alte PC + 4. A vermischt alten und neuen Registerwert; D unterstellt eine automatische Sicherung.

9. A, D — A und D: s0 ist callee-saved; der weitere jal überschreibt ra, daher muss die äußere Rückkehradresse anderweitig erhalten werden. Ein Stackslot ist eine mögliche Sicherung. B ist falsch, weil t-Register caller-saved sind. ret springt nur und führt keinen Epilog aus.

10. A, C — A und C: Die ALU bildet die effektive Adresse aus rs1 und Immediate. Der Speicherinhalt dieser Adresse wird in rd geschrieben. B verwechselt Datenquelle und Ziel; D beschreibt einen Store statt eines Loads.

11. B, C, D — B, C und D: Gleiche Operanden erzeugen bei der Subtraktion null. PCSrc $=$ Branch$\wedge$Zero wählt das Ziel aus altem PC und fertigem Byteoffset. A addiert die vier Byte des sequenziellen Pfads fälschlich zum genommenen Branchziel.

12. B — Nur B: Beide Befehle sind Registeroperationen mit RegWrite = 1 und demselben Opcode. Die Detailsteuerung wertet die gültigen Funktionsfelder aus. ALUControl ist eine interne Kurscodierung und kein Instruktionsfeld der ISA.

13. B, C — B und C: MemtoReg wählt nur die Rückschreibquelle für die Registerbank; RegWrite sperrt deren Übernahme. X bezeichnet in der Tabelle ein Don't Care, keinen elektrischen Pegel. Wird RegWrite fälschlich eins, kann die gewählte Quelle einen zusätzlichen Registerwert verändern.

14. A, C, D — A, C und D: Die Zeit ist $N \cdot \text{CPI} \cdot T$, also 400 ns beziehungsweise 200 ns. Der Speedup von B gegenüber A beträgt 2. B ist als Aussage falsch: Ein kleinerer CPI allein genügt ohne Vergleich der Taktperiode nicht.

15. A — Nur A: Vor der ersten Ausführung wird N vorhergesagt, dann T gespeichert. Die zweite Vorhersage T ist richtig; die dritte Vorhersage T ist falsch und speichert N. Es gibt zwei Fehler. B ignoriert die erste Aktualisierung, C übersieht den zweiten Treffer, D verwechselt alte Vorhersage und neuen Zustand.

16. A, D — A und D: Die arithmetische Rechtsverschiebung wiederholt das gesetzte Vorzeichenbit und liefert `0xF0000001`. Die logische Variante `srli` füllt mit Nullen und ergibt `0x10000001`, weshalb B falsch ist. Bei einem registergesteuerten RV32I-Shift zählen nur die unteren fünf Bits des Betrags; `35` hat dort den Wert drei. C verwendet fälschlich alle Bits.

17. B, C — B und C: Ein Load berechnet in EX erst die Adresse und liefert den geladenen Wert nach MEM. Die unmittelbar folgende Subtraktion wartet einen Takt und nutzt den Wert danach über MEM/WB. Die leere Fünfstufenpipeline benötigt für zwei Instruktionen ohne Zusatzkosten sechs, mit Load-Use sieben Takte. A verwechselt Adresse und Daten; D zählt die ungültige Hardware-Bubble als Programminstruktion.

18. B, C, D — B, C und D: `add` erzeugt elf, `addi` daraus 15. Die nachfolgende Subtraktion braucht den jüngeren Wert 15 aus EX/MEM und berechnet $15 - 3 = 12$. Der ältere Wert elf kann gleichzeitig in MEM/WB stehen, darf aber nicht gewinnen. A verletzt die sequenzielle Befehlswirkung.

19. A — Nur A: `addi` liest hier nur `x0`; der zweite Rechenoperand ist das Immediate fünf. Die Bits 24–20 gehören beim I-Format zu dieser Konstanten und benennen kein `rs2`. B und D erfinden eine Registerquelle; C folgert einen Stall aus einem unbenutzten Bitfenster. Der Load schreibt `t0`, doch der Folgebefehl verwendet dessen Wert nicht.

20. A, B — A und B: Die zwei jüngeren Einträge gehören zum falschen Pfad. Ihr Flush muss die Zustandswirkungen verhindern, bevor der Store den Speicher oder `addi` ein Register verändert. Verworfene Einträge erhöhen die abgeschlossene Befehlszahl N nicht, können aber Zeit kosten; C ist falsch. D beschreibt Halten statt Verwerfen und verwechselt damit Stall und Flush.

Aufgabe 2: Zahlendarstellung, Logik und Register (10 Punkte)

a) Stellen Sie −37 als achtbittigen Zweierkomplementwert in Binär- und Hexadezimalschreibweise dar. Zeigen Sie den Rechenweg ausgehend von +37. (3 Punkte)

b) Addieren Sie die achtbittigen Muster `0111 1111` und `0000 0001`. Geben Sie Ergebnis und hinausfallenden Übertrag an. Liegt bei signed Interpretation ein Überlauf vor? Begründen Sie. (2 Punkte)

c) Ein Einbit-Multiplexer wählt bei $S = 0$ den Eingang A, bei $S = 1$ den Eingang B. Geben Sie die Boolesche Gleichung für Y an und füllen Sie die vier Tabellenzeilen aus. (3 Punkte)

S	A	B	Y
0	1	0	
1	1	0	
1	0	1	
0	0	1	

d) Ein positiv flankengetriggertes Einbitregister startet mit $Q = 0$. Setup und Hold sind eingehalten. Geben Sie Q nach jedem der beiden aufeinanderfolgenden Ereignisse an; die Schreibfreigabe ist jeweils eins. (2 Punkte)

Ereignis	Dateneingang D	Ausgang Q danach
Steigende Taktflanke	1	
Takt bleibt hoch, danach ändert sich D	0	

Musterlösung und Bewertung

a) $37 = 32 + 4 + 1$: `0010 0101`. Invertieren: `1101 1010`; eins addieren: `1101 1011`, also `0xDB`. Ein Punkt für +37, ein Punkt für die korrekte Bildung des negativen Musters, ein Punkt für den Hexwert.

b) Ergebnis `1000 0000`, hinausfallender Übertrag null. Als signed Rechnung sollen zwei nichtnegative Werte $127 + 1 = 128$ entstehen; das achtbittige Ergebnis stellt jedoch −128 dar. Damit liegt signed Überlauf vor. Ein Punkt für Ergebnis und Übertrag zusammen (je 0,5), ein Punkt für Überlauf mit Begründung (je 0,5).

c) $Y = (\overline{S} \wedge A) \vee (S \wedge B)$. Ergebnisse von oben nach unten: 1, 0, 1, 0. Ein Punkt für eine äquivalente Gleichung, je 0,5 Punkte pro Tabellenwert.

d) Nach der steigenden Flanke $Q = 1$, beim dauerhaft hohen Takt weiterhin $Q = 1$. Die zweite Änderung erzeugt keine neue aktive Flanke. Je ein Punkt pro Wert.

Aufgabe 3: Array und Schleife in RV32I (16 Punkte)

Die Blattfunktion `sum_first` soll die ersten `n` Elemente eines Arrays addieren und die Summe zurückgeben. Für `n=0` liefert sie null, ohne ein Arrayelement zu lesen.

Es gilt $0 \leq n \leq 4$. Ab `0x3000` liegen vier 32-Bit-Zweierkomplementwerte lückenlos: −2, 5, 7, −3. Bei Eintritt enthält `a0` die Basisadresse `0x3000`, `a1` die vorzeichenlose Länge `n`. Ein Element mit Index `i` steht vier mal `i` Byte hinter der Basis; die Indizes beginnen bei null. Die Funktion benötigt keine Unteraufrufe und darf ohne Stackframe auskommen.

a) Schreiben Sie eine vollständige RV32I-Blattfunktion `sum_first` mit Schleife und Rückkehr. Verwenden Sie `t0` für die unveränderte Arraybasis, `t1` für den Index und `t2` für die Summe; `t3`–`t5` dürfen Hilfswerte halten. Nutzen Sie für die Länge einen unsigned Vergleich und prüfen Sie die Grenze vor jedem Load. Verwenden Sie echte Instruktionen; als einzige Pseudoinstruktion ist `ret` erlaubt. Das Ergebnis gehört in `a0`. (12 Punkte)

Musterlösung zu a) (12 Punkte)

```
sum_first:
    addi t0, a0, 0
    addi t1, x0, 0
    addi t2, x0, 0
sum_loop:
    bgeu t1, a1, sum_end
    slli t3, t1, 2
    add t4, t0, t3
    lw t5, 0(t4)
    add t2, t2, t5
    addi t1, t1, 1
    jal x0, sum_loop
sum_end:
    addi a0, t2, 0
    jalr x0, 0(ra)
```

Bewertung: Basis erhalten 1, Index null 1, Summe null 1, korrekter unsigned Abbruch vor dem Load 2, Byteoffset und Adresse 2, Wort laden 1, Summe aktualisieren 1, Index erhöhen 1, Rücksprung zur Prüfung 1, Ergebnis und Rückkehr zusammen 1 Punkt (je 0,5). Entsprechende funktional gleichwertige Lösungen erhalten dieselben Punkte. Ein Stackframe ist unnötig: keine Unteraufrufe, keine Veränderung callee-saved Register.

Aufgabe 3: Ergebnisse und Grenzen

b) Bestimmen Sie für $\texttt{n=0}$ und $\texttt{n=3}$ jeweils den Rückgabewert und die Zahl der Array-Loads. (2 Punkte)

c) Geben Sie die Byteadresse und den gespeicherten Zahlenwert des Elements mit Index zwei an. Begründen Sie die Skalierung des Index in der Adressrechnung. (2 Punkte)

Musterlösung zu b) und c)

b) $\texttt{n=0}$: Ergebnis 0, kein Array-Load. $\texttt{n=3}$: Ergebnis $-2 + 5 + 7 = 10$, drei Array-Loads. Je 0,5 Punkte für die vier Angaben.

c) $A_2 = \texttt{0x3000} + 4 \cdot 2 = \texttt{0x3008}$; Wert 7. Ein Punkt für die Adresse, 0,5 Punkte für den Wert. Ein Element belegt vier Byte und der Speicher ist byteadressiert; deshalb entspricht Index zwei einem Abstand von acht Byte. 0,5 Punkte für diese Begründung. Die Adresse ist nicht der Inhalt des Elements.

Korrekturhinweis: Ein Grenzfehler wird am fehlerhaften Prüfungsteil bewertet. Nachfolgende korrekte Rechen- und Adressierungsarbeit bleibt separat bewertbar; ein Folgefehler wird nicht mehrfach abgezogen.

Aufgabe 4: Funktionsaufruf und Stacktrace (10 Punkte)

Vor `PC=0x1000` gilt `a0=4`, `s0=99`, `sp=0x2000`. Der Stackbereich ist beschreibbar. Alle Zeilen sind echte Vier-Byte-Instruktionen; die angegebenen Adressen und Labels gelten für diesen Papierausschnitt.

```
0x1000: jal ra, F
0x1004: addi t0, a0, 0
F:
0x1100: addi sp, sp, -16
0x1104: sw ra, 12(sp)
0x1108: sw s0, 8(sp)
0x110C: addi s0, a0, 0
0x1110: jal ra, Double
0x1114: add a0, a0, s0
0x1118: lw s0, 8(sp)
0x111C: lw ra, 12(sp)
0x1120: addi sp, sp, 16
0x1124: jalr x0, 0(ra)
Double:
0x1200: slli a0, a0, 1
0x1204: jalr x0, 0(ra)
```

a) Tragen Sie den Zustand unmittelbar **nach** der jeweiligen Instruktion ein. `PC danach` bezeichnet die als Nächstes auszuführende Adresse. Geben Sie `a0` und `s0` dezimal, die übrigen Größen hexadezimal an. (5 Punkte)

gerade ausgeführt	PC danach	a0	s0	ra	sp
0x1110					
0x1124					

b) Geben Sie für beide Stores die effektive Adresse und das dort abgelegte 32-Bit-Wort in Hexadezimalschreibweise an. (2 Punkte)

c) Warum wird `ra` gesichert? Warum beträgt der Frame trotz nur zwei gesicherter Wörter 16 Byte? (2 Punkte)

d) Nur die Instruktion `lw ra, 12(sp)` wird durch eine wirkungslose Instruktion ersetzt; alle angegebenen Adressen bleiben unverändert. Welche Adresse wählt dann der erste Sprung bei `0x1124`? (1 Punkt)

Musterlösung und Bewertung

gerade ausgeführt	PC danach	a0	s0	ra	sp
0x1110	0x1200	4	4	0x1114	0x1FF0
0x1124	0x1004	12	99	0x1004	0x2000

a) Je 0,5 Punkte pro Tabellenzelle. Double verdoppelt vier zu acht; F addiert den erhaltenen Eingang vier, Ergebnis zwölf.

b) Store bei 0x1104: Adresse 0x1FFC, Wort 0x00001004. Store bei 0x1108: Adresse 0x1FF8, Wort 0x00000063. Je 0,5 Punkte pro Adresse beziehungsweise Wort.

c) Der innere jal überschreibt ra mit 0x1114; F braucht später aber seine äußere Rückkehradresse 0x1004. Ein Punkt. Zwei Wörter belegen acht Byte; der 16-Byte-Frame erhält die vereinbarte Stackausrichtung während der gesamten Funktion. Ein Punkt. ret würde weder Werte laden noch sp korrigieren.

d) Ziel 0x1114 statt 0x1004, weil die innere Linkadresse in ra bleibt. Ein Punkt.

Aufgabe 5: Datenpfad und Steuerwerk (12 Punkte)

Verwenden Sie den Single-Cycle-Kursausschnitt für `add`, `sub`, `and`, `or`, `lw`, `sw` und `beq`. `ALUSrc` wählt 0 für die zweite Registerquelle oder 1 für das Immediate; `MemtoReg` wählt 0 für das ALU-Ergebnis oder 1 für Speicherlesedaten. Register- und Speicherschreiben haben getrennte Freigaben.

Interne Kurscodes: `ALUOp` = `00` für ADD, `01` für SUB-Vergleich, `10` für Auswertung der Register-Funktionsfelder. `ALUControl` = `0010` für ADD, `0110` für SUB. ImmGen liefert bei Branches den fertigen vorzeichenerweiterten Byteoffset; danach erfolgt kein weiterer Shift.

a) Füllen Sie die Hauptsteuerzeilen mit 0, 1 oder X (Don't Care). Bei einer unerheblichen Auswahl sind auch begründete konkrete Werte zulässig. (6 Punkte)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`lw`						
`beq`						

b) Geben Sie `ALUOp` und `ALUControl` für `lw` und `beq` an. (2 Punkte)

c) Ein `beq` steht bei `PC=0x4000`. Beide Quellen enthalten 13; ImmGen liefert −12. Bestimmen Sie `Zero`, `PCSrc` und den nächsten PC. (3 Punkte)

d) Ein anderer Load `lw t0, 12(t1)` liest mit `t1=0x2000` ein Speicherwort mit Wert 99. Durch einen Steuerfehler ist nur `MemtoReg=0` statt 1. Welchen Wert erhält `t0` und warum? (1 Punkt)

Musterlösung und Bewertung

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`lw`	1	1	1	1	0	0
`beq`	0	X	0	0	0	1

a) Je 0,5 Punkte pro Zelle. Bei beq sind MemtoReg = 0 und = 1 ebenfalls korrekt, weil RegWrite = 0 die Übernahme sperrt. Andere Freigaben sind hier keine Don't Cares.

b) `lw`: `ALUOp=00`, `ALUControl=0010`. `beq`: `ALUOp=01`, `ALUControl=0110`. Je 0,5 Punkte pro Code.

c) $13-13=0$: `Zero=1`. PCSrc $=$ Branch $\wedge$ Zero $=1$. Neuer PC: $\texttt{0x4000}-12=\texttt{0x3FF4}$. Je ein Punkt. Das Branchziel wird aus dem alten PC gebildet.

d) `t0` erhält `0x200C`, also dezimal 8204. Der MUX wählt wegen MemtoReg = 0 die von der ALU berechnete Adresse statt des gelesenen Wortes 99. RegWrite = 1 übernimmt diese falsche Quelle. Je 0,5 Punkte für Wert und Begründung.

Aufgabe 6: Pipeline und Laufzeit (12 Punkte)

Fünf Stufen: IF (Befehl holen), ID (decodieren und Register lesen), EX (rechnen), MEM (Datenzugriff), WB (Rückschreiben / Abschluss). Anfangs leere Pipeline, höchstens ein neuer Befehl pro Takt, getrennte Befehls- und Datenzugriffe; jeder Load braucht eine MEM-Stufe. Forwarding aus EX/MEM und MEM/WB und Load-Use-Erkennung sind vorhanden. Der jüngste passende Produzent ist maßgeblich; ist sein Wert noch nicht bereit, muss gewartet werden. Ein Schreibversuch auf `x0` erzeugt keine Forwardingquelle. Ein Load liefert seine Daten erst nach MEM. Der Stall hält PC und IF/ID und erzeugt eine Bubble in ID/EX; ältere Arbeit läuft weiter. Am Ende von WB geschriebene Registerwerte sind erst im Folgetakt in ID lesbar; ein gleichzeitiger WB→ID-Bypass fehlt.

Vor dem Ausschnitt steht `s0=0x3000`; dort liegt das Wort 7:

```
lw   t0, 0(s0)
addi t0, t0, 5
add  a0, t0, t0
```

a) Tragen Sie die Stufenfolge ein und kennzeichnen Sie den Stall. Wiederholtes ID beziehungsweise IF mit Stall-Markierung ist zulässig. (5 Punkte)

Befehl / Takt	1	2	3	4	5	6	7	8
`lw t0, 0(s0)`								
`addi t0, t0, 5`								
`add a0, t0, t0`								

b) Begründen Sie den Stall und erklären Sie, warum zwischen `addi` und `add` kein weiterer Stall nötig ist. (2 Punkte)

c) Welche Werte enthalten `t0` und `a0` nach Abschluss? (1 Punkt)

d) Ein **anderer** anfangs leerer Pipeline-Lauf umfasst 100 gültig abgeschlossene Maschineninstruktionen, acht Load-Use-Stallzyklen und drei genommene Branches. Die Branch-Auflösung erfolgt in EX; jeder genommene Branch kostet zwei zusätzliche Takte. Die Zusatzkosten überschneiden sich nicht, weitere Stalls gibt es nicht. Die Taktperiode beträgt 2 ns. Bestimmen Sie Gesamttakte, endlichen CPI, Gesamtzeit und Speedup gegenüber einer Ausführung derselben 100 Instruktionen mit CPI eins und 8 ns Taktperiode. Gezählt wird vom ersten IF bis Ende des letzten WB; Bubbles und verworfene Einträge zählen nicht zu den 100 Instruktionen. (4 Punkte)

Musterlösung und Bewertung

Befehl / Takt	1	2	3	4	5	6	7	8
`lw`	IF	ID	EX	MEM	WB			
`addi`		IF	ID	ID*	EX	MEM	WB	
`add`			IF	IF*	ID	EX	MEM	WB

a) • bedeutet gehalten; in Takt 4 liegt eine wirkungslose Bubble in EX. Ein Punkt für den vollständigen Load-Durchlauf, zwei für den addi-Durchlauf mit passendem Stall (je ein Punkt für Stufenlage und Halt), zwei für den entsprechend verschobenen add-Durchlauf (je ein Punkt für Stufenlage und Halt). „STALL“ anstelle von ID* beziehungsweise IF* ist gleichwertig.

b) Das Loadwort ist vor Ende von Takt 4 noch nicht verfügbar und erreicht addi in EX bei Takt 5 aus MEM/WB. Ein Punkt. Das ALU-Ergebnis von addi liegt bereits nach dessen EX vor und wird in Takt 6 aus EX/MEM an beide benutzten Quellen von add weitergereicht. Ein Punkt.

c) `t0=12`, `a0=24`. Je 0,5 Punkte.

d) $C = N + 4 + S + 2B = 100 + 4 + 8 + 2 \cdot 3 = 118$ Takte. CPI $= 118/100 = 1{,}18$. $t = 118 \cdot 2\,\text{ns} = 236\,\text{ns}$. Vergleichszeit $100 \cdot 1 \cdot 8\,\text{ns} = 800\,\text{ns}$, Speedup $800/236 \approx 3{,}39$. Je ein Punkt pro gefragter Größe. Der CPI enthält hier Füllen, Leeren und Zusatzkosten; die übrigen Teilgrößen werden bei plausibler Weiterrechnung mit einem zuvor falschen C nicht erneut abgezogen.

Aufgabe 7: Maschinencode und Bytefolge (10 Punkte)

Es gelten RV32I, Little Endian und die gemeinsamen Annahmen. Die unten angegebenen Namen `x11` und `x14` bezeichnen Registernummern, nicht deren Inhalte. Ein Referenzblatt darf verwendet werden.

I-Format von Bit 31 nach Bit 0: `imm[11:0]`, `rs1`, `funct3`, `rd`, `opcode`.

S-Format von Bit 31 nach Bit 0: `imm[11:5]`, `rs2`, `rs1`, `funct3`, `imm[4:0]`, `opcode`.

Für `addi` gelten `funct3=000`, `opcode=0010011`; für `sw` gelten `funct3=010`, `opcode=0100011`.

Die Formate haben von links nach rechts die Feldbreiten I: 12, 5, 3, 5, 7 Bit beziehungsweise S: 7, 5, 5, 3, 5, 7 Bit.

a) Codieren Sie `addi x14, x11, -28`. Tragen Sie alle Felder binär ein und geben Sie das vollständige 32-Bit-Maschinenwort hexadezimal an. (4 Punkte)

imm[11:0]	rs1	funct3	rd	opcode

b) Codieren Sie `sw x14, -36(x11)`. Tragen Sie alle Felder binär ein und geben Sie das vollständige 32-Bit-Maschinenwort hexadezimal an. (3 Punkte)

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode

c) Das Instruktionswort aus a) liegt ab Byteadresse `0x5400`. Geben Sie die vier Bytewerte mit ihren Adressen an. (2 Punkte)

d) Eine ältere Instruktion schreibt `x4`. In den Bits 24–20 des `addi` aus a) steht ebenfalls `00100`. Begründet allein diese Übereinstimmung eine Register-Datenabhängigkeit? Erklären Sie anhand des Formats. (1 Punkt)

Musterlösung und Bewertung

a) Das zwölfbittige Zweierkomplement ist $4096 - 28 = 4068$, also `0xFE4` = `1111 1110 0100`.

imm[11:0]	rs1	funct3	rd	opcode
111111100100	01011	000	01110	0010011

Maschinenwort `0xFE458713`. Ein Punkt für das Immediate, ein Punkt für beide Registernummern (je 0,5), ein Punkt für die korrekte Feldanordnung mit `funct3` und Opcode, ein Punkt für das zusammengesetzte Hexwort.

b) Das zwölfbittige Zweierkomplement ist $4096 - 36 = 4060$, also `0xFDC` = `1111 1101 1100`. Die oberen sieben Bits sind `1111110`, die unteren fünf `11100`. Der Offset wird in Byte codiert, nicht durch vier geteilt.

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
1111110	01110	01011	010	11100	0100011

Maschinenwort `0xFCE5AE23`. Ein Punkt für die beiden Immediate-Teile (je 0,5), ein Punkt für die beiden Registerfelder mit richtiger Feldanordnung und den vorgegebenen Funktions-/Opcode-Feldern, ein Punkt für das Hexwort. `sw` hat kein Zielregisterfeld `rd`; `rs2` benennt die Datenquelle.

c) `0x5400`: `0x13`, `0x5401`: `0x87`, `0x5402`: `0x45`, `0x5403`: `0xFE`. Je 0,5 Punkte. Little Endian legt das niederwertigste Byte an der kleinsten Adresse ab; die Feldpositionen innerhalb des Instruktionsworts ändern sich dadurch nicht.

d) Nein. Beim I-Format gehören Bits 24–20 zum Immediate und sind kein `rs2`-Feld. Dieses `addi` liest nur `x11`; allein aus dem Schreiben von `x4` entsteht daher keine Abhängigkeit. Ein Punkt für die begründete Verneinung.

Korrekturhinweis: Ein falsch gebildetes Immediate wird dort bewertet. Korrekte Feldanordnung und Hexzusammensetzung aus diesem Muster bleiben getrennt bewertbar. In c) wird die Little-Endian-Zerlegung des in a) angegebenen Maschinenworts bewertet; ein dortiger Codierfehler wird nicht erneut abgezogen.

Themenabdeckung und Zeitrahmen

Diese Übersicht dient der Korrektur und enthält keine zusätzlichen Aufgaben.

Einheit	Inhalte in den Multiple-Choice-Fragen	Vertiefung auf Papier
1	1–3, 6: Abstraktion, Bitmuster, Logik, Speicherung, Byteordnung	Aufgabe 2
2	1, 4–5, 16, 19: Register, Immediates, Codierung, Shifts	Aufgaben 3 und 7
3	5–7, 11: Erweiterung, Formate, Speicher, Branches	Aufgaben 3, 5 und 7
4	8–9: Sprung, Link, ABI und Registererhaltung	Aufgaben 3–4
5	10–11: Adressrechnung, Rückschreiben, Branchpfad	Aufgabe 5
6	11–13: Hauptsteuerung, ALU-Steuerung, Don’t Care	Aufgabe 5
7	14–15, 17–20: Laufzeit, CPI, Vorhersage, Forwarding, Stall, Flush	Aufgaben 6 und 7d

Geplante Verteilung: 20 Minuten für 20 Auswahlfragen; 70 Minuten für sechs Papieraufgaben mit 70 Punkten. Die Punkte folgen dem Arbeitsaufwand und der Zahl eigenständiger Schritte. Kleine Einzelangaben erhalten entsprechend halbe oder Viertelpunkte. Der Zeitrahmen ist eine didaktische Schätzung für vorbereitete Studierende, keine mit einer Studierendengruppe gemessene Bearbeitungszeit.

Bei Papieraufgaben werden äquivalente fachlich richtige Lösungen akzeptiert. Folgefehler werden nur am ersten fehlerhaften Schritt abgezogen, soweit die Weiterrechnung die übrigen gefragten Fähigkeiten zeigt. Der Zeitrahmen und die Hilfsmittel dieser Probeklausur legen keine abweichenden Regeln einer späteren regulären Prüfung fest.